

Propuesta de Proyecto — Modelos Estocásticos

VoxelScribe: Detección Probabilística de Tinta en Papiros del Vesuvius Challenge

Capítulo 18 — Probabilistic Programming · AIMA 4th ed. · Russell & Norvig

1. Resumen ejecutivo

Proponemos implementar un motor de inferencia basado en **Relational Probability Models (RPMs)** con **Gibbs Sampling** para detectar tinta en fragmentos de papiro del **Vesuvius Challenge** (scrollprize.org). El Vesuvius Challenge es un concurso científico activo cuyo objetivo es leer rollos de papiro carbonizados por la erupción del Vesubio en el año 79 DC, usando tomografía computarizada (CT) de alta resolución.

El proyecto aplica directamente los conceptos del Capítulo 18: cada píxel de la imagen CT es una variable aleatoria en un RPM instanciado (grounding), cuya etiqueta —¿hay tinta aquí?— se infiere mediante Gibbs Sampling sobre un Markov Random Field. Se usa el fragmento público **Frag1** del dataset EduceLab-Scrolls, que incluye imágenes CT y una máscara de tinta como ground truth.

Cobertura del capítulo 18: aproximadamente 80%, muy por encima del 50% requerido.

2. El problema físico

En el año 79 DC, la erupción del Vesubio enterró la biblioteca de la Villa de los Papiros en Herculano. Los rollos quedaron carbonizados — preservados pero imposibles de abrir a mano sin destruirlos. Desde 2023, el Vesuvius Challenge financia la lectura virtual de estos rollos mediante escáneres CT de alta resolución.

El escáner CT produce un cubo de números 3D donde cada número representa la densidad del material en ese punto del espacio. El papiro y la tinta de carbono tienen densidades casi idénticas, lo que hace que la detección de tinta sea extremadamente difícil con métodos simples de umbralización.

Los fragmentos del dataset tienen una ventaja: están físicamente abiertos y fueron fotografiados con luz infrarroja, lo que permitió crear máscaras de tinta manuales (ground truth). Esto los convierte en el dataset ideal para entrenar y evaluar algoritmos de detección.

3. Los datos — qué son y de dónde vienen

Se usan exactamente **dos archivos** del Fragmento 1 (Frag1), descargables públicamente sin registro:

Archivo 1 — Imagen CT

URL:

https://dl.ash2txt.org/fragments/Frag1.volpkg/working/54keV_exposed_surface/surface_volume/layers/32.tif

Formato: imagen TIFF de 16 bits.

Contenido: array 2D donde cada valor es la intensidad del escáner CT en ese píxel (0–65535). Valores más altos = material más denso = píxel más brillante.

Archivo 2 — Máscara de tinta

URL:

https://dl.ash2txt.org/fragments/Frag1.volpkg/working/54keV_exposed_surface/inklabels.png

Formato: imagen PNG binaria.

Contenido: array 2D donde cada valor es 0 (no hay tinta) o 1 (hay tinta). Fue creado por humanos usando fotografía infrarroja. Es el *ground truth* contra el que se compara el algoritmo. Ambos archivos se cargan en Python como arrays NumPy. No se requieren librerías adicionales para la carga más allá de PIL para leer los formatos de imagen.

4. El modelo — conexión con el Capítulo 18

4.1 Tipos y variables (§18.1 — Relational Probability Models)

Tipo: Pixel, indexado por coordenada (i, j)

Función observada:

Intensity(i, j) → valor entero 0–65535 [VISIBLE en los datos]

Función oculta (lo que queremos inferir):

HasInk(i, j) → {0, 1} [DESCONOCIDA, objetivo de la inferencia]

4.2 Grounding (§18.1.3)

Para una imagen de H×W píxeles, el grounding del RPM instancia H×W variables HasInk, una por cada píxel. Si se trabaja con una región de 300×300 píxeles, el modelo grounded tiene **90.000 variables aleatorias** conectadas en una red de dependencias locales. Este es el paso de "unrolling" descrito en el libro.

4.3 Dependencias

Cada variable $\text{HasInk}(i,j)$ depende de dos factores:

Factor 1 — Potencial unario (programa generativo, §18.4):

- $P(\text{Intensity}(i,j) \mid \text{HasInk}(i,j) = 1) \sim \text{Gaussiana}(\mu_{\text{ink}}, \sigma_{\text{ink}})$
- $P(\text{Intensity}(i,j) \mid \text{HasInk}(i,j) = 0) \sim \text{Gaussiana}(\mu_{\text{noink}}, \sigma_{\text{noink}})$

Las medias y desviaciones estándar se aprenden de los datos usando la máscara de tinta como supervisión en un subconjunto de entrenamiento.

Factor 2 — Potencial pairwise (coherencia espacial):

La tinta forma trazos continuos. Un píxel con tinta tiende a estar rodeado de otros con tinta. Esto se modela como $\exp(\beta \times \text{número_de_vecinos_con_mismo_valor})$, donde β es un hiperparámetro que controla la fuerza de la coherencia espacial.

4.4 Inferencia: Gibbs Sampling (§18.1.3)

Para cada píxel (i,j) , se calcula la distribución condicional $P(\text{HasInk}(i,j) \mid \text{Markov_blanket}(i,j))$. El Markov blanket de cada píxel son sus cuatro vecinos (arriba, abajo, izquierda, derecha) y su propia intensidad observada. Esta distribución se calcula como el producto de los dos factores, normalizado a probabilidades que suman 1, y luego se samplea el nuevo valor.

Un **sweep** completo actualiza todos los píxeles en orden aleatorio. Se ejecutan sweeps de burn-in (descartados) seguidos de sweeps de muestreo (usados para estimar el posterior).

5. Algoritmo completo paso a paso

PASO 0 — Aprender las distribuciones:

Usar la máscara de entrenamiento para estimar μ_{ink} , σ_{ink} , μ_{noink} , σ_{noink} .

PASO 1 — Inicializar:

$\text{HasInk}(i,j) = 0$ para todos los píxeles.

PASO 2 — Sweep de Gibbs:

Para cada píxel (i,j) en orden aleatorio:

score_tinta = $P(\text{Intensity}|\text{tinta}) \times \exp(\beta \times \text{vecinos_con_tinta})$

score_noink = $P(\text{Intensity}|\text{sin_tinta}) \times \exp(\beta \times \text{vecinos_sin_tinta})$

$p = \text{score_tinta} / (\text{score_tinta} + \text{score_noink})$

$\text{HasInk}(i,j) \sim \text{Bernoulli}(p)$

PASO 3 — Repetir:

Ejecutar N_{burn} sweeps de burn-in (descartar). Luego N_{sample} sweeps acumulando conteos.

PASO 4 — Posterior:

$P(\text{HasInk}(i,j) = 1 \mid \text{datos}) = \text{conteo_de_unos} / N_{\text{sample}}$

PASO 5 — Decisión:

$\text{HasInk_final}(i,j) = 1$ si posterior > 0.5 , sino 0.

PASO 6 — Evaluación:

Comparar HasInk_final contra inklabels.png . Reportar: Accuracy, Precision, Recall, F1.

6. Escenarios de prueba

Escenario 1 — $\beta = 0$ (solo intensidad, sin vecinos)

Configuración: $\beta = 0$. Cada píxel decide únicamente basándose en su propia intensidad CT, ignorando a sus vecinos. Equivale a un clasificador bayesiano ingenuo por píxel.

Propósito: línea base. Permite medir cuánto mejora el modelo al agregar coherencia espacial.

Resultados esperados: accuracy moderada, precision baja (muchos falsos positivos aislados), recall moderada.

Escenario 2 — $\beta = 1.5$ (coherencia espacial moderada)

Configuración: $\beta = 1.5$. Los vecinos influyen pero no dominan la decisión.

Propósito: caso normal. Verificar que la coherencia espacial mejora precision respecto al Escenario 1.

Análisis adicional: graficar la energía del MRF por iteración para verificar convergencia del Gibbs Sampling. Si la energía baja y se estabiliza, la cadena convergió.

Escenario 3 — $\beta = 5.0$ (sobre-suavizado)

Configuración: $\beta = 5.0$. Los vecinos dominan completamente la decisión.

Propósito: mostrar el efecto negativo del sobre-suavizado. Con β muy alto, el modelo tiende a crear regiones uniformes grandes, borrando los detalles finos de las letras.

Resultados esperados: precision alta pero recall muy baja — el modelo es conservador y "ve"

menos tinta de la que hay.

7. Cobertura del Capítulo 18

Sección del libro	Cómo aparece en el proyecto
§18.1 — Relational Probability Models	Tipo Pixel, funciones HasInk e Intensity
§18.1.3 — Grounding	Instanciar HasInk(i,j) para cada píxel de la imagen
§18.1.3 — Gibbs Sampling	Algoritmo principal del proyecto
§18.4 — Programas generativos	$P(\text{Intensity} \mid \text{HasInk})$ como distribución gaussiana
§18.4.3 — Inferencia en programas generativos	Proceso completo de burn-in y muestreo del posterior

Cobertura total estimada: ~80% del capítulo.

8. Restricciones técnicas

- **Lenguaje:** Python 3.10+
- **Librería de cómputo (única permitida):** NumPy
- **Librería adicional (solicitar permiso al profe):** PIL/Pillow — solo para leer archivos TIF/PNG, no realiza ningún cómputo del modelo.
- **NO se usará:** PyMC, Stan, pgmpy, scikit-learn, scipy.stats, ni ninguna librería que implemente MCMC o inferencia en redes bayesianas.

Todo el Gibbs Sampler, el cálculo de distribuciones gaussianas, el potencial pairwise y las métricas de evaluación se implementan **desde cero con NumPy**.

9. División de trabajo (5 personas, 6 días)

Día 1 — Todos juntos

Leer §18.1 y §18.4 del libro. Ver los datos. Que todos entiendan el modelo antes de separarse a trabajar.

Día 2 — En paralelo

- **Persona 1:** Marco teórico y diagrama UML.

- **Persona 2:** Descarga y preprocesamiento de datos, separación train/test.
- **Persona 3:** Clases del RPM + aprendizaje de distribuciones gaussianas.
- **Persona 4:** Gibbs Sampler completo con burn-in y seguimiento de energía.
- **Persona 5:** Métricas de evaluación y estructura del informe.

Día 3 — Integración

- **Personas 2, 3, 4:** Conectar todos los módulos en un script principal y correr la primera prueba completa.
- **Personas 1, 5:** Documentar funciones, avanzar en el informe.

Día 4 — Escenarios 1 y 2

- **Personas 3, 4:** Correr y registrar resultados de los escenarios 1 y 2.
- **Personas 1, 5:** Redactar análisis de resultados en el informe.

Día 5 — Escenario 3 y manual técnico

- **Personas 3, 4:** Correr Escenario 3 y armar tabla comparativa de los tres escenarios.
- **Persona 2:** Manual técnico y manual de usuario.

Día 6 — Cierre

- **Personas 1, 5:** Ensamblar informe final, conclusiones, revisión.
- **Personas 2, 3, 4:** Prueba de reproducibilidad desde cero siguiendo el manual de usuario.

10. Entregables

- Marco teórico (RPMs, Gibbs Sampling, MRFs)
- Descripción y justificación del problema
- Diseño de la aplicación (diagrama UML de clases y flujo)
- Código fuente completo (Python + NumPy únicamente)
- Manual de usuario y manual técnico
- Documentación de los 3 escenarios de prueba con análisis de resultados

11. Título y resumen para el correo al profesor

Título: VoxelScribe: Detección Probabilística de Tinta en Fragmentos de Papiro Herculanense mediante Relational Probability Models y Gibbs Sampling

Resumen: Este proyecto aplica los Relational Probability Models (RPMs) del Capítulo 18 de Artificial Intelligence a Modern Approach al problema de detección de tinta en fragmentos de papiro del Vesuvius Challenge, un concurso científico activo dedicado a leer virtualmente rollos carbonizados del siglo I DC. Se usa el fragmento público Frag1 del dataset EduceLab-Scrolls, que incluye imágenes de tomografía computarizada (CT) y una máscara de tinta creada

mediante fotografía infrarroja como ground truth. Cada píxel de la imagen CT es modelado como una variable aleatoria $HasInk$ en un RPM instanciado (grounding, §18.1.3). La inferencia se realiza mediante Gibbs Sampling sobre un Markov Random Field donde el potencial unario corresponde a un modelo generativo gaussiano de $P(Intensity|HasInk)$ (§18.4), y el potencial pairwise modela la coherencia espacial de los trazos de tinta. Se implementan desde cero, usando exclusivamente Python y NumPy: el modelo probabilístico, el Gibbs Sampler completo con burn-in configurable, el seguimiento de convergencia por energía, y las métricas de evaluación (accuracy, precision, recall, F1). Se documentan tres escenarios de prueba variando el parámetro β de coherencia espacial (0, 1.5 y 5.0), mostrando el trade-off entre sensibilidad individual por píxel y suavizado espacial. La cobertura del Capítulo 18 es aproximadamente del 80%.